Lab 6

Sam Kutsyn, 2581500 EE 446

July 31, 2026

Edge Impulse Results

You can access the project at https://studio.edgeimpulse.com/studio/1074054.

Raw inputs: State the raw inputs used in the Edge Impulse project.

> 3-axis accelerometer data (accX, accY, accZ), sampled at 100 Hz with a 2000 ms window size and 220 ms window stride (zero-padding enabled).

NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

> Spectral Features - 33 total input features generated from the accelerometer axes. Examples: accX RMS, accX Peak 1 Height, accX Peak 1 Freq, accY RMS, accY Peak 1 Height.

NN classifier outputs: State the outputs of the NN classifier.

> 2 output classes: nominal and off (fan operating status).

Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

> Only 4 of the 33 available features were selected for the K-means anomaly detector: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0–5.0 Hz, accZ Spectral Power 2.0–5.0 Hz.

Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

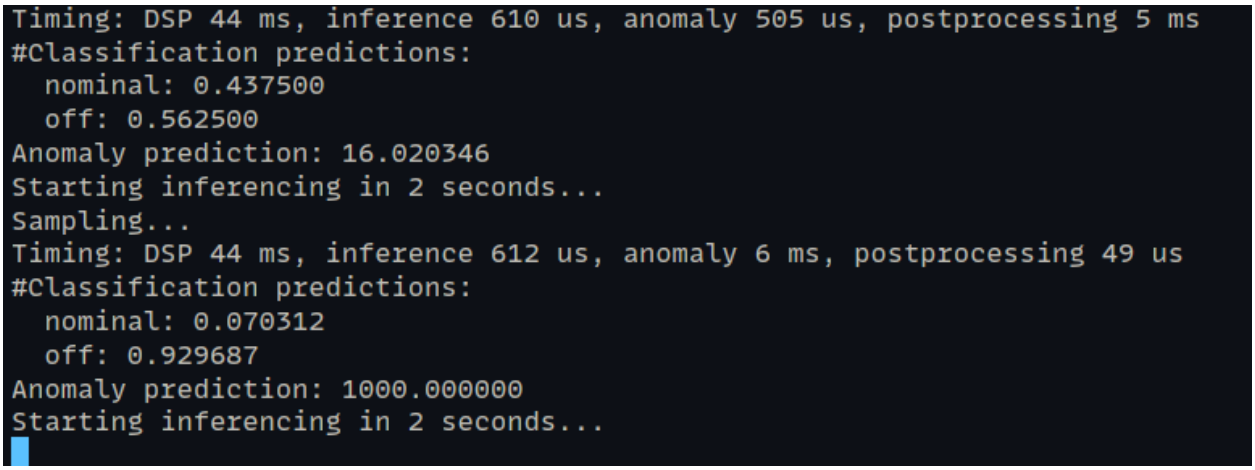


Figure 1: Arduino serial monitor output

Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

The classifier reliably distinguished "nominal" vs. "off" states in testing, reflecting the strong separation seen in the spectral features. However, its output shouldn't be trusted in all conditions - it was trained on a narrow feature set (RMS and peak/spectral power in specific frequency bands) from one fan setup, so performance may degrade with different fan models or mounting positions not represented in training. The anomaly detector, using an even smaller 4-feature subset, is useful for catching novel behavior outside these two classes, but its sensitivity depends heavily on how representative the K-means clusters are of true "normal" operation.

Autoencoder Model Results

Training and Validation Loss Curves

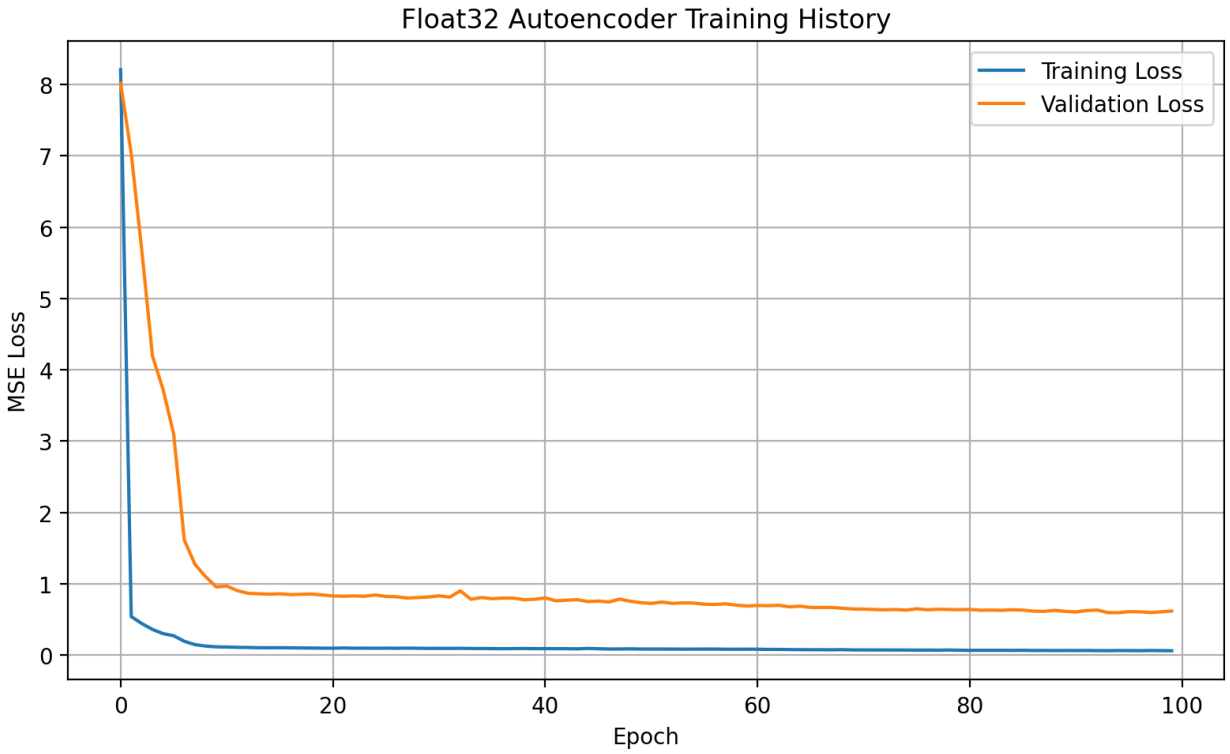


Figure 2: Float32 training history

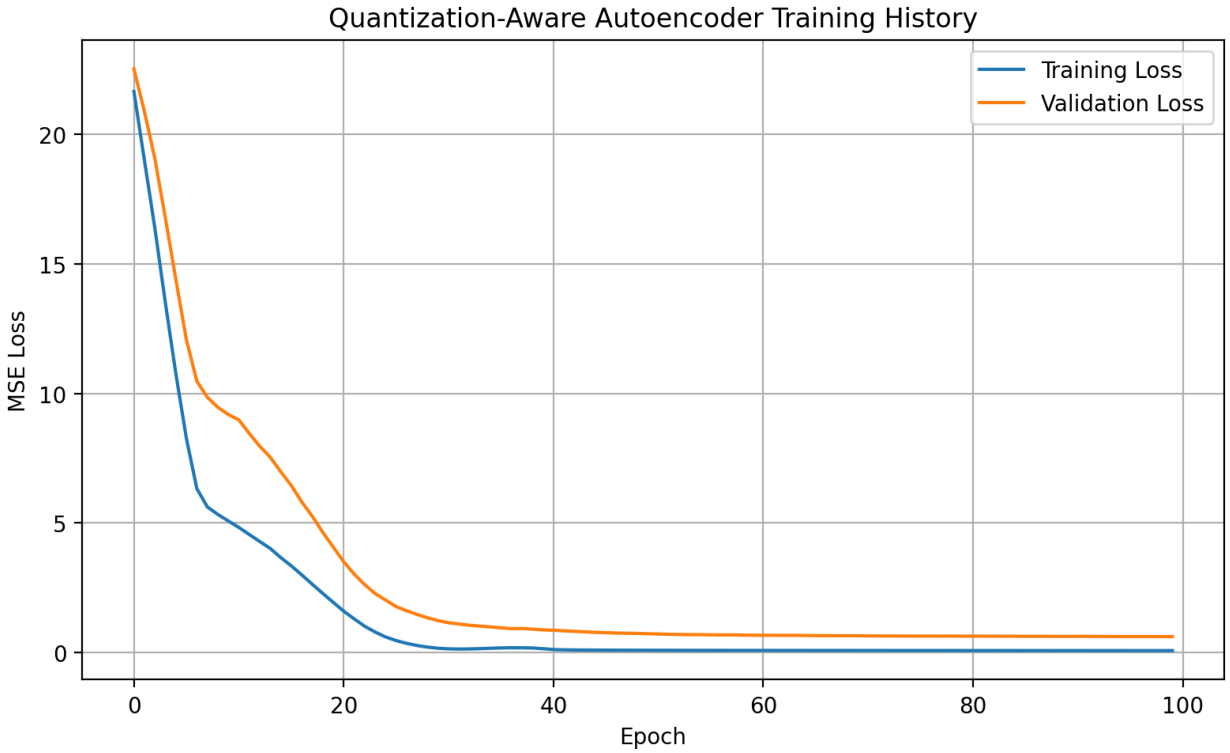


Figure 3: QAT training history

PCA Visualization

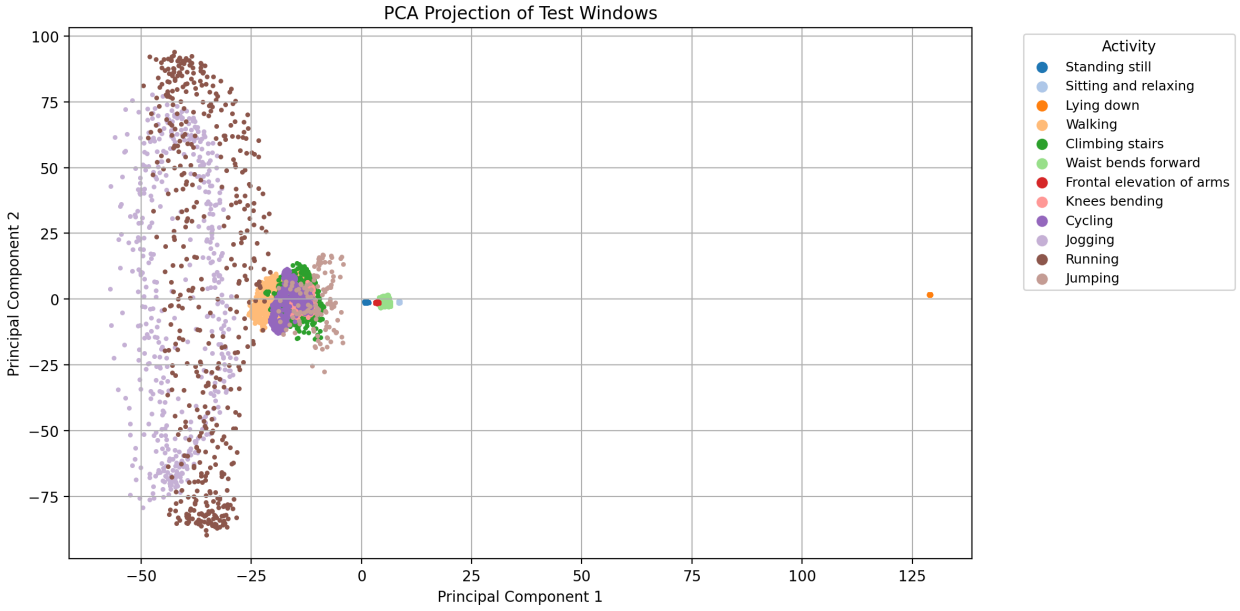


Figure 4: PCA projection

t-SNE Visualization

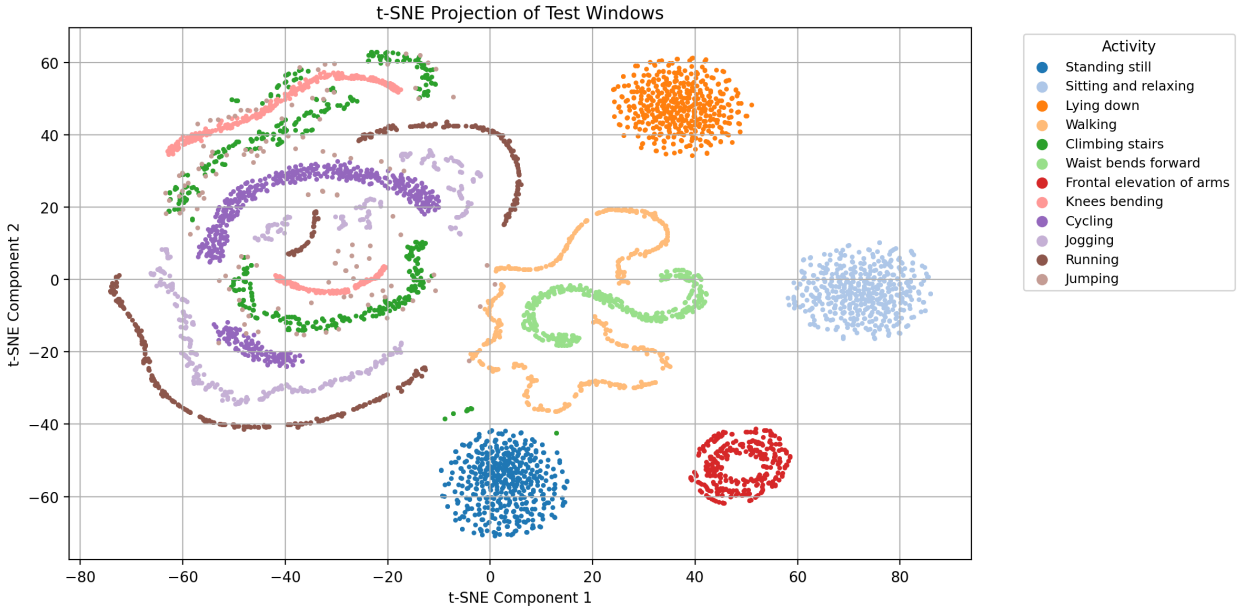


Figure 5: t-SNE projection

UMAP Visualization

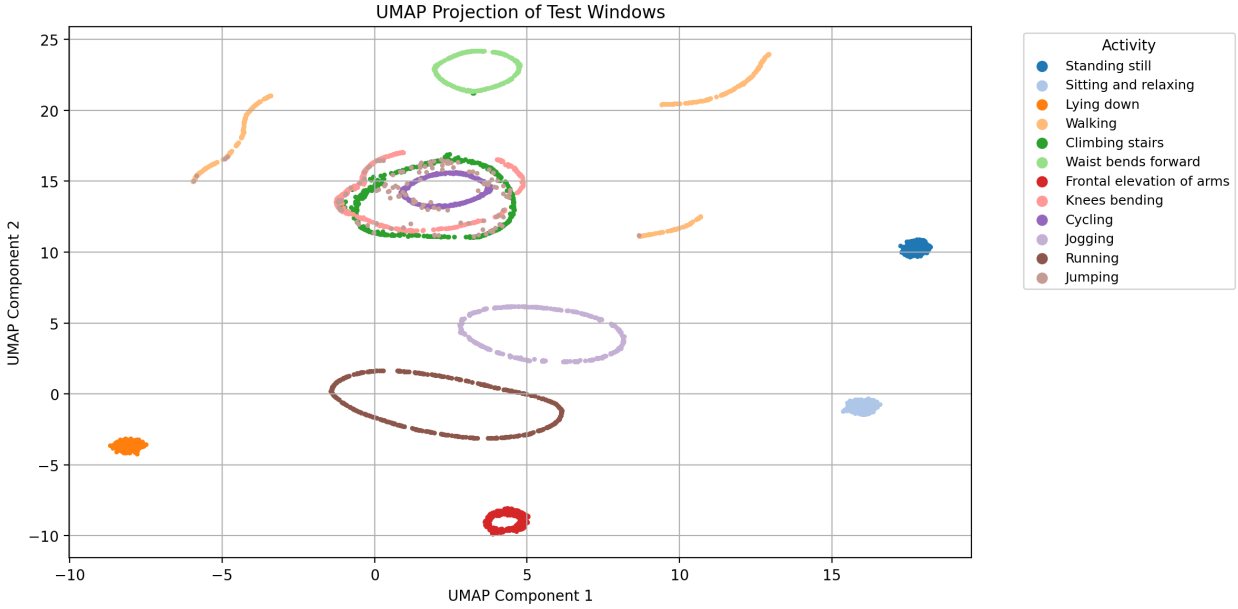


Figure 6: UMAP projection

Reconstruction Error Distribution

Figure 7: Reconstruction error before threshold

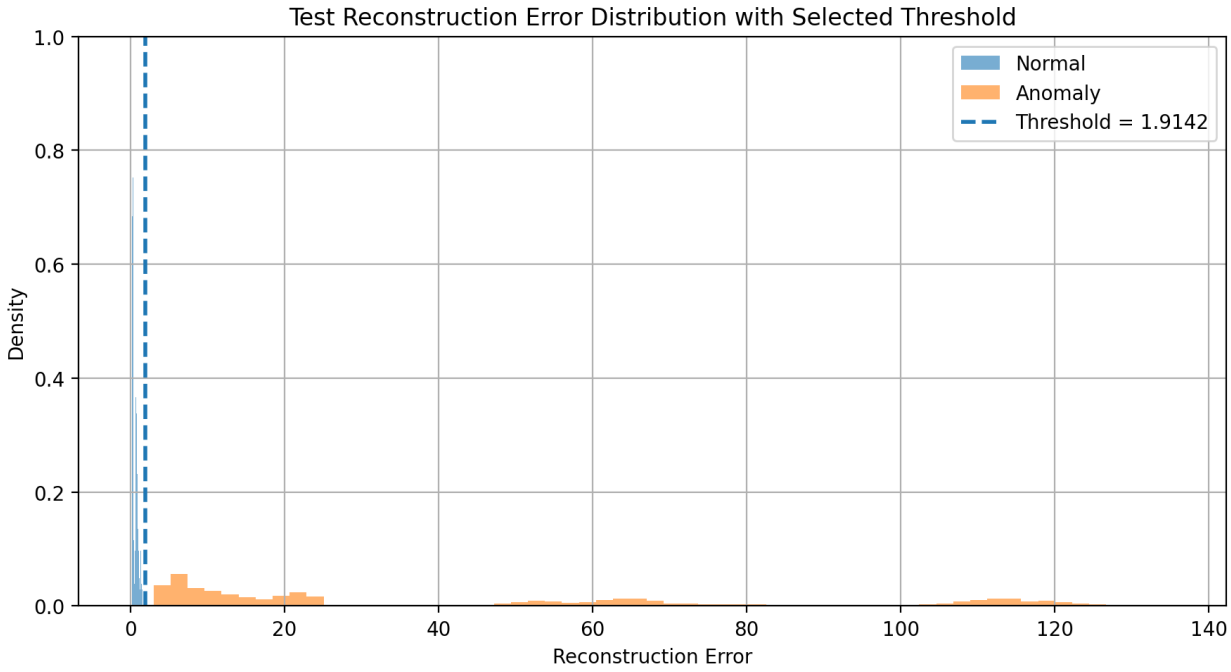
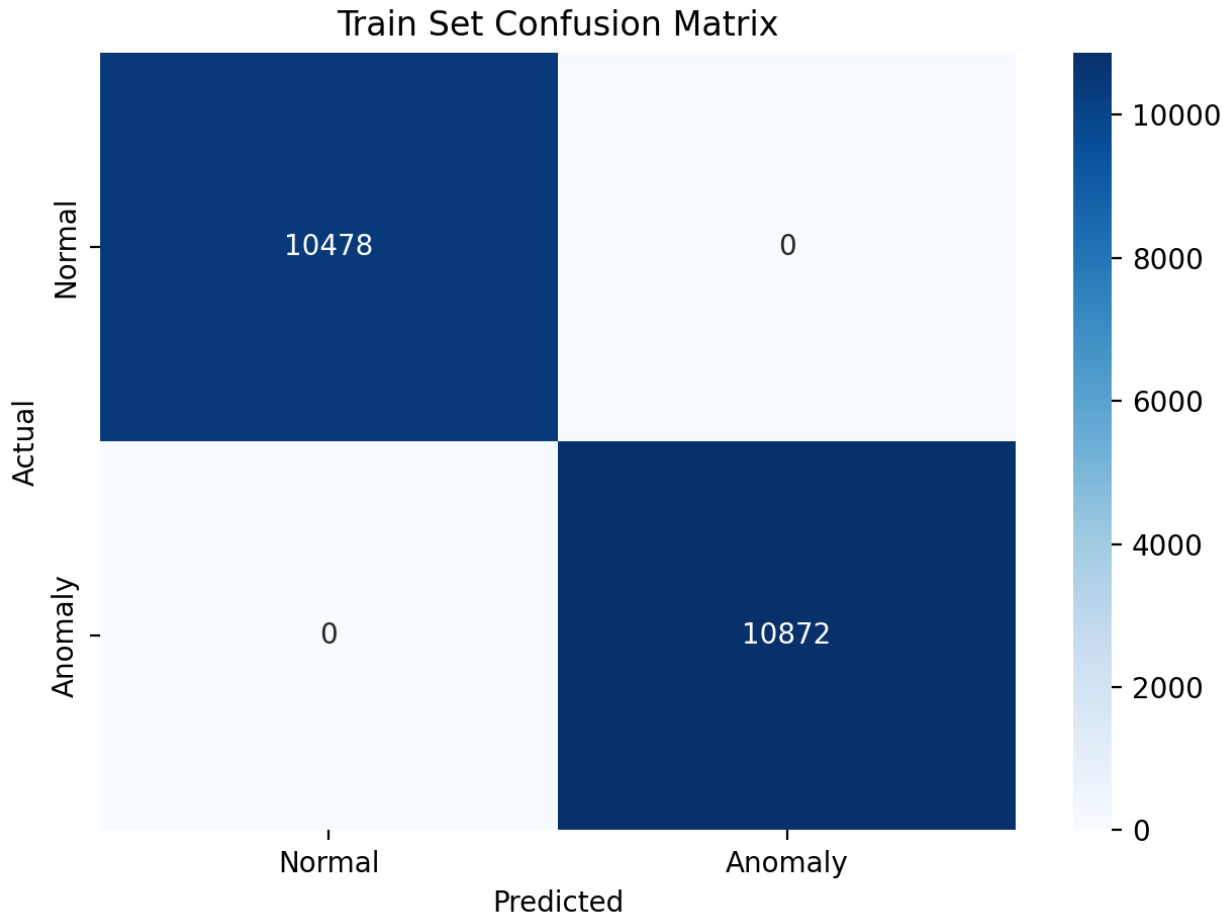
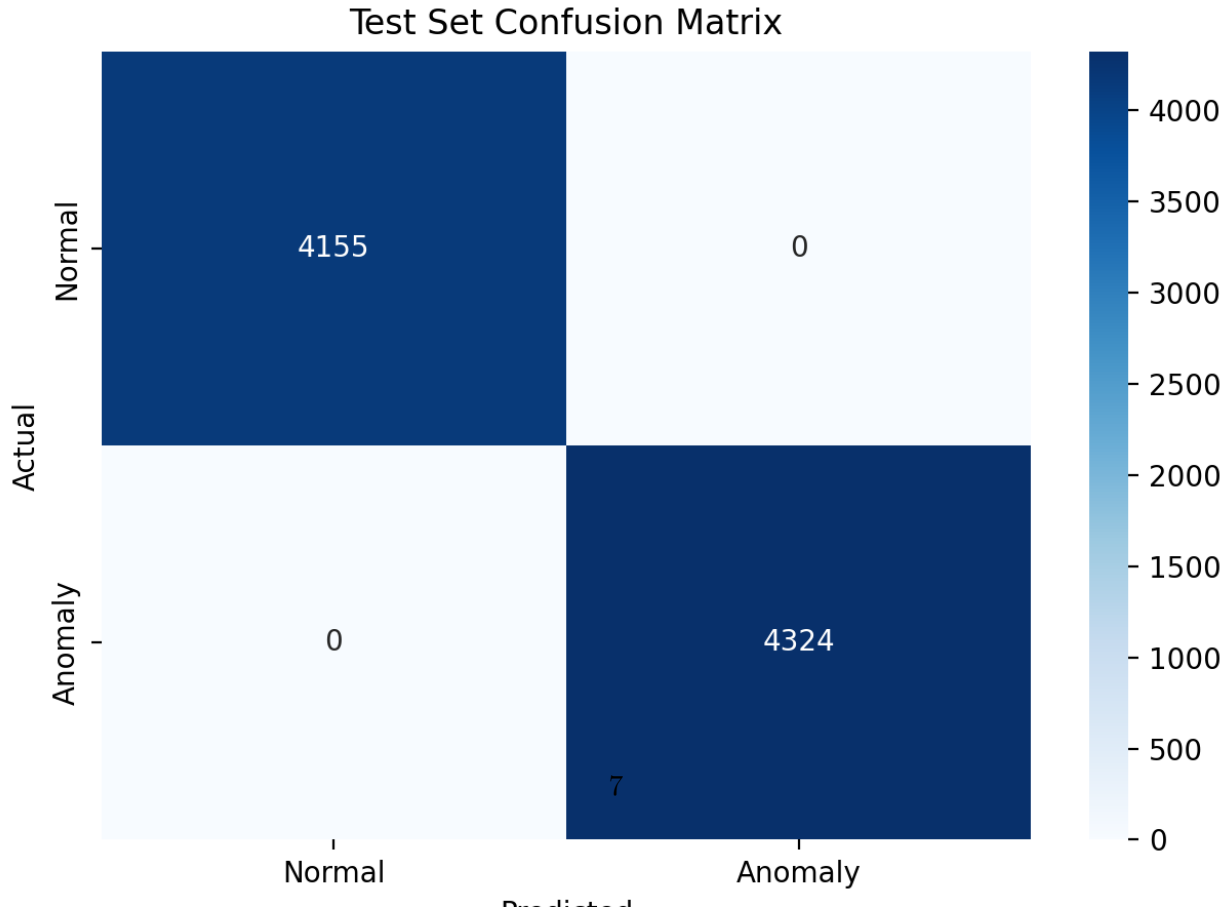


Figure 8: Reconstruction error with threshold

Confusion Matrix





Classification Report / Evaluation Metrics

Train set (quantization-aware model):

Class	Precision	Recall	F1-score	Support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
Accuracy			1.00	21350

Test set (quantization-aware model):

Class	Precision	Recall	F1-score	Support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
Accuracy			1.00	8479

Int8 quantized model results matched the float32/QAT results exactly on both train and test sets.

Arduino Serial Monitor Output

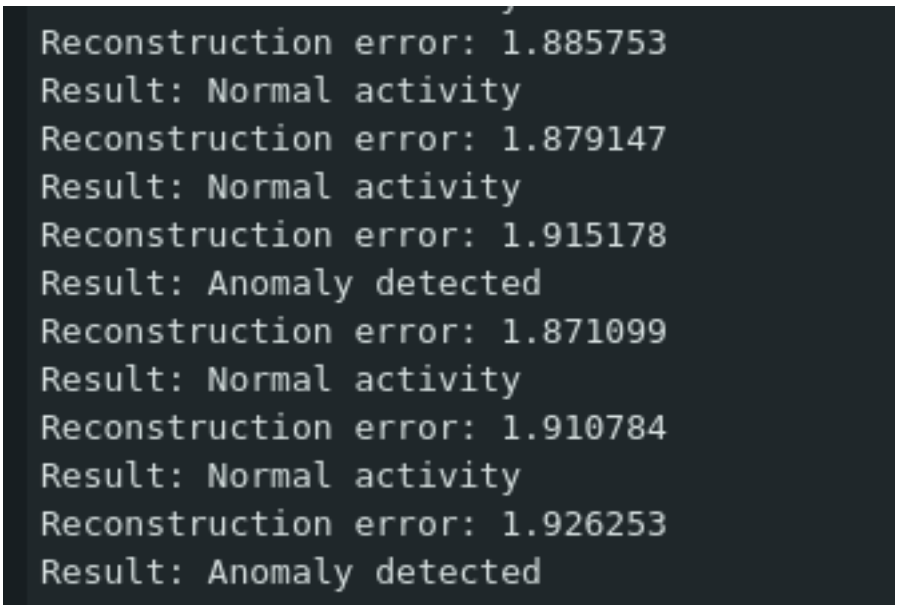


Figure 9: Arduino serial monitor output

Discussion Questions

What threshold did you choose for anomaly detection, and why?

I used $mean + 9 \times std$ of the normal training reconstruction errors, giving a threshold of 1.9142. The normal and anomaly error distributions were extremely well separated, so any multiplier in a wide range would have worked.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder was trained only to reconstruct normal activity windows, so it learns a compressed representation specific to that data's patterns. When given an anomalous (physically different) window, it reconstructs it poorly, producing a much higher mean-squared error. Thresholding this error lets us flag windows the model can't represent well as anomalies.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The int8 quantization parameters (input/output scale and zero-point), the window size and feature ordering used during training, and the selected reconstruction-error threshold.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

If the sketch samples, orders, or quantizes data differently than training, the input distribution shifts and the model's learned reconstruction behavior no longer applies, making the anomaly threshold invalid.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The model can only detect anomalies as "different from training data," not the specific type of anomaly. Its accuracy depends heavily on how representative the training data (and quantization calibration set) was of real-world use.

Submission

See submission files in ./submission folder.